

NOTAS IMPORTANTES

Duración: 2 horas

Con material: No

Serán factores relevantes en la valoración de las soluciones desarrolladas, la claridad en la presentación, el estilo y la metodología de trabajo, acordes a los impartidos en el curso. Recordar la utilización de nombres autoexplicativos para funciones, variables, constantes, etc.

Ejercicio 1. (5 puntos)

- A. Explicar en qué consisten las **una de las optimizaciones** de MFSet (disjoint-set).
- B. Cuando se utiliza el MFSet en Kruskal ¿se lo hace de tamaño V o A? **Justificar.**

Ejercicio 2. (10 puntos)

Un coleccionista está armando una aplicación para gestionar las figuritas del Mundial que tiene en su poder. La comunicación entre coleccionistas se vuelve difícil porque cada uno se refiere a las figuritas de forma distinta:

- Algunos por **nombre y apellido del jugador** (asumir que la combinación es única).
- Otros por el **número de figurita** (entero entre 0 y 1023).
- Otros por la combinación **nacionalidad + número de camiseta** (asumir que también es única).

Se quiere que la aplicación responda **cuántas copias tiene** el coleccionista de una figurita según cualquiera de los tres criterios, de la **manera más eficiente** posible dados los temas vistos en el curso.

Se pide

- A. ¿Qué estructura(s) de datos utilizaría para soportar las consultas por los tres criterios? **Justificar.**

- B. **Implementar** las siguientes operaciones e **indicar su orden temporal**, asumiendo la representación diseñada en la parte A:

- a. **agregarFigurita**: dados los atributos de una figurita, la **agrega al sistema** con cantidad 1; si ya existía, incrementa su cantidad en 1.

void **agregarFigurita**(string nombre, string apellido, int numeroFigurita, string nacionalidad, int numeroCamiseta)

- b. **cuantasTengo**: devuelve la **cantidad de copias** de la figurita identificada por nombre y apellido. Si no la tiene, devuelve 0.

int **cuantasTengo**(string nombre, string apellido)

- c. **cambio**: que registra que el coleccionista entregó una copia de **doy** y recibió una copia de **recibo**. Asumir que **doy** es una figurita repetida (cantidad ≥ 2). Si **recibo** ya existía en la colección, incrementa su cantidad; si no, la agrega.

void **cambio**(Figurita doy, Figurita recibo)

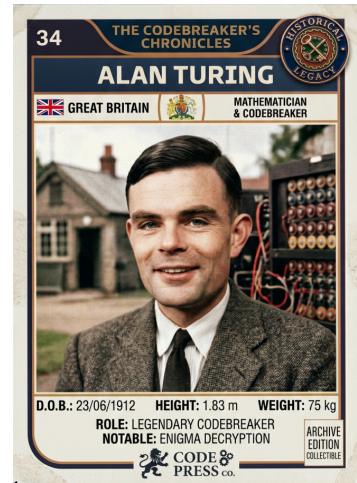
```
struct Figurita {  
    string nombre;  
    string apellido;  
    int numeroFigurita;  
    string nacionalidad;  
    int numeroCamiseta;  
};
```

Ejercicio 3. (15 puntos)

Un estudiante de Estructuras de Datos se descargó el álbum digital "Grandes de la Computación": una aplicación de figuritas con personajes históricos como Turing, Dijkstra, Bellman, Floyd, Hoare, Knuth, Prim y Kruskal, entre otros. Cada figurita está identificada por **un número entero** (empezando del 1) y existen **264** figuritas.

Aunque actualmente no tiene ninguna figurita, **su único objetivo** es conseguir la figurita de **Turing (fig 34)**.

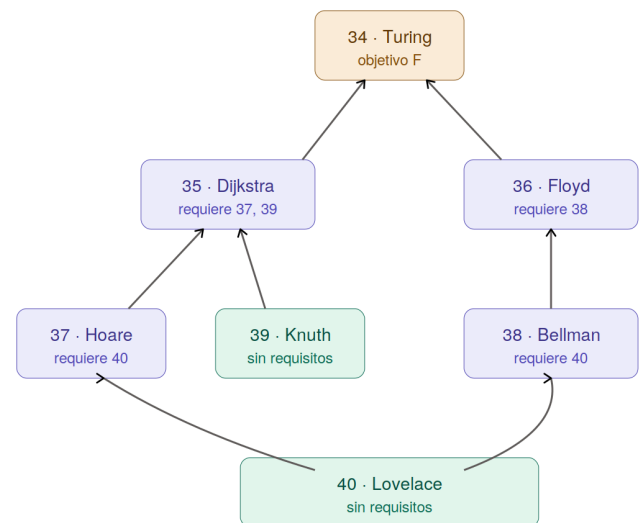
La app permite intercambiar figuritas entre coleccionistas, pero el coleccionista que tiene a Turing **solo la cambia por un conjunto específico de figuritas**; cada una de esas, a su vez, está en manos de otro coleccionista que también pone condiciones, y así sucesivamente. Algunos coleccionistas podrían no pedir nada a cambio.



Como el álbum es digital, cada figurita obtenida **puede clonarse** dentro de la app y usarse en **tantos intercambios como se quiera**. Es decir, una vez obtenida una figurita, queda disponible para todos los cambios posteriores que la requieran, sin importar cuántos sean.

En el grafo de la derecha se puede ver un ejemplo de que figuritas necesita/quiere cada coleccionista.

- ¿Qué **características** tiene el grafo en caso de que **tenga solución**? ¿Cuáles tiene en caso de que **no** la tenga?
- ¿Qué **algoritmo** utilizaría para saber si existe solución o no? Explique brevemente **cómo lo utilizaría**.
- Implemente** la siguiente función que retorna uno de los órdenes posibles en los cuáles se pueden dar los intercambios.



List<int>* **intercambios**(Grafo*g, int F)

Notas (parte C):

- Puede asumir que siempre hay solución.
- Puede usar TADs y estructuras auxiliares sin implementarlos.
- Un posible retorno es una lista en este orden: 39,40,38,36,37,35,34.
- **Si bien existen otras figuritas NO deben tenerse en cuenta para la lista de retorno (por ejemplo a 1, 5 ó 255).** Puede asumir que la componente débilmente conexa que contiene a F son los únicos vértices que tienen aristas dentro del grafo.